main.py

```python
coins = [1, 5, 10, 25]
amount = 63

coins.sort(reverse=True)

result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins:", *result)
print("Minimum coins =", len(result))
```

```
Coins: 25 25 10 1 1 1
Minimum coins = 6


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
weights = [10, 20, 30]
profits = [60, 100, 120]
capacity = 50

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

items.sort(reverse=True)

max_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        max_profit += profit
    else:
        max_profit += ratio * capacity
        break

print("Maximum profit =", max_profit)
```

```
Maximum profit = 240.0


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
weights = [5, 10, 15]
profits = [30, 40, 45]
capacity = 20

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

items.sort(reverse=True)

max_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        max_profit += profit
    else:
        max_profit += ratio * capacity
        break

print("Maximum profit =", max_profit)
```

```
Maximum profit = 85.0


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
jobs = ['P1', 'P2', 'P3', 'P4']
deadlines = [1, 1, 2, 2]
profits = [40, 30, 20, 10]

job_list = []

for i in range(len(jobs)):
    job_list.append((profits[i], deadlines[i], jobs[i]))

job_list.sort(reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0

for profit, deadline, job in job_list:
    for j in range(deadline - 1, -1, -1):
        if slots[j] is None:
            slots[j] = job
            total_profit += profit
            break

print("Selected jobs =", *[job for job in slots if job])
print("Maximum profit =", total_profit)
```

```
Selected jobs = P1 P3
Maximum profit = 60


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
jobs = ['A', 'B', 'C', 'D']
deadlines = [2, 1, 2, 1]
profits = [100, 19, 27, 25]

job_list = []

for i in range(len(jobs)):
    job_list.append((profits[i], deadlines[i], jobs[i]))

job_list.sort(reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0

for profit, deadline, job in job_list:
    for j in range(deadline - 1, -1, -1):
        if slots[j] is None:
            slots[j] = job
            total_profit += profit
            break

print("Selected jobs =", *[job for job in slots if job])
print("Maximum profit =", total_profit)
```

```
Selected jobs = C A
Maximum profit = 127


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
import heapq

characters = ['a', 'b', 'c', 'd', 'e', 'f']
frequencies = [5, 9, 12, 13, 16, 45]
heap = []
for i in range(len(characters)):
    heapq.heappush(heap, (frequencies[i], characters[i]))
while len(heap) > 1:
    freq1, char1 = heapq.heappop(heap)
    freq2, char2 = heapq.heappop(heap)

    new_node = (char1 + char2)

    heapq.heappush(heap, (freq1 + freq2, new_node))

root = heap[0][1]

codes = {}

def generate_codes(node, code=""):
    if len(node) == 1:
        codes[node] = code
        return

    mid = len(node) // 2

    generate_codes(node[:mid], code + "0")
    generate_codes(node[mid:], code + "1")

generate_codes(root)
print("Huffman Codes:")
for char in characters:
    print(char + ":", codes[char])
```

```
Huffman Codes:
a: 10
b: 110
c: 010
d: 011
e: 111
f: 00


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
jobs = ['A', 'B', 'C', 'D', 'E']
deadlines = [2, 1, 2, 1, 3]
profits = [100, 50, 10, 20, 30]

job_list = []

for i in range(len(jobs)):
    job_list.append((profits[i], deadlines[i], jobs[i]))

job_list.sort(reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0

for profit, deadline, job in job_list:
    for j in range(deadline - 1, -1, -1):
        if slots[j] is None:
            slots[j] = job
            total_profit += profit
            break

print("Selected jobs =", *[job for job in slots if job])
print("Maximum profit =", total_profit)
```

```
Selected jobs = B A E
Maximum profit = 180


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
import heapq
characters = ['x', 'y', 'z', 'w']
frequencies = [7, 8, 14, 25]
heap = []
for i in range(len(characters)):
    heapq.heappush(heap, (frequencies[i], characters[i]))
while len(heap) > 1:
    freq1, char1 = heapq.heappop(heap)
    freq2, char2 = heapq.heappop(heap)

    new_node = char1 + char2

    heapq.heappush(heap, (freq1 + freq2, new_node))
root = heap[0][1]

codes = {}

def generate_codes(node, code=""):
    if len(node) == 1:
        codes[node] = code
        return

    mid = len(node) // 2

    generate_codes(node[:mid], code + "0")
    generate_codes(node[mid:], code + "1")

generate_codes(root)

print("Huffman Codes:")

for char in characters:
    print(char + ":", codes[char])
```

```
Huffman Codes:
x: 10
y: 11
z: 01
w: 00


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
weights = [4, 8, 2, 6]
profits = [20, 40, 14, 30]
capacity = 10

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

items.sort(reverse=True)

max_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        max_profit += profit
    else:
        max_profit += ratio * capacity
        break

print("Maximum profit =", max_profit)
```

```
Maximum profit = 54.0


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
weights = [2, 3, 5, 7]
profits = [10, 5, 15, 7]
capacity = 8

items = []

for i in range(len(weights)):
    ratio = profits[i] / weights[i]
    items.append((ratio, weights[i], profits[i]))

items.sort(reverse=True)

max_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        max_profit += profit
    else:
        max_profit += ratio * capacity
        break

print("Maximum profit =", max_profit)
```

```
Maximum profit = 26.666666666666668


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```
coins = [1, 2, 5, 10]
amount = 28

coins.sort(reverse=True)

result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins:", *result)
print("Minimum coins =", len(result))
```

```
Coins: 10 10 5 2 1
Minimum coins = 5


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
import heapq

characters = ['A', 'B', 'C', 'D']
frequencies = [10, 20, 30, 40]

heap = []

for i in range(len(characters)):
    heapq.heappush(heap, (frequencies[i], characters[i]))

while len(heap) > 1:
    freq1, char1 = heapq.heappop(heap)
    freq2, char2 = heapq.heappop(heap)
    new_node = char1 + char2
    heapq.heappush(heap, (freq1 + freq2, new_node))
root = heap[0][1]
codes = {}
def generate_codes(node, code=""):
    if len(node) == 1:
        codes[node] = code
        return

    mid = len(node) // 2

    generate_codes(node[:mid], code + "0")
    generate_codes(node[mid:], code + "1")

generate_codes(root)

print("Huffman Codes:")

for char in characters:
    print(char + ":", codes[char])
```

```
Huffman Codes:
A: 01
B: 10
C: 11
D: 00


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
coins = [1, 2, 5]
amount = 11

coins.sort(reverse=True)

result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins:", *result)
print("Minimum coins =", len(result))
```

```
Coins: 5 5 1
Minimum coins = 3


...Program finished with exit code 0
Press ENTER to exit console.
```

```python
jobs = ['J1', 'J2', 'J3', 'J4', 'J5']
deadlines = [2, 1, 2, 1, 3]
profits = [20, 15, 10, 5, 1]

job_list = []

for i in range(len(jobs)):
    job_list.append((profits[i], deadlines[i], jobs[i]))

job_list.sort(reverse=True)

max_deadline = max(deadlines)
slots = [None] * max_deadline

total_profit = 0

for profit, deadline, job in job_list:
    for j in range(deadline - 1, -1, -1):
        if slots[j] is None:
            slots[j] = job
            total_profit += profit
            break

print("Selected jobs =", *[job for job in slots if job])
print("Maximum profit =", total_profit)
```

```
Selected jobs = J2 J1 J5
Maximum profit = 36


...Program finished with exit code 0
Press ENTER to exit console.
```

main.py

```python
coins = [1, 3, 7, 10]
amount = 24

coins.sort(reverse=True)

result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins:", *result)
print("Minimum coins =", len(result))
```

```
Coins: 10 10 3 1
Minimum coins = 4


...Program finished with exit code 0
Press ENTER to exit console.
```